

Федеральное государственное автономное образовательное учреждение высшего
образования

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ

«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Высшая школа бизнеса

Введение в инженериию данных

Проект

Работу выполнила:

Алрида Лана Окба БАСБ 252

Евсеев Никита Артемович БАСБ 252

Петросян Каролина Эдуардовна БАСБ 252

Чекмаева Софья Алексеевна БАСБ 252

Шитова Марина БАСБ 252

Москва 2025

Содержание

Схема данных.....	3
Обоснование декомпозиции модели данных.....	3
Реляционная схема данных.....	5
DDL скрипты.....	5
Основные этапы парсинга.....	8
Загрузка данных.....	13
Витрины.....	13
Витрина заказов.....	13
Витрина товаров.....	15

Схема данных

Обоснование декомпозиции модели данных

В текущем задании нам представлена модель данных заказов и доставки этих заказов для клиентов. Для каждой таблицы представлена следующая логика деления:

1. **Таблица users** - Таблица хранит в себе информацию о пользователях, которые когда-то совершили заказ. Для каждого пользователя мы уже имеем идентификатор и номер телефона. Так как формат номера телефона может быть разным, то храним его в виде строки.
2. **Таблица driver** - По информации о водителе мы имеем аналогичный набор данных. В отдельной таблице храним идентификатор водителя как первичный ключ и номер телефона.
3. **Таблица store** - Для каждого уникального магазина храним идентификатор и адрес магазина. Для работы с данными адреса, адрес декомпозируется на название магазина (store_name), город(store_city), котором находится магазин и отдельно адрес (store_address).

Каждый заказ содержит в себе информацию о товарах в заказе, а также информацию о доставке. Один товар может быть указан в нескольких заказах и заказ хранит более одного товара. Таким образом приходим к наличию связи “Многие ко многим” между заказом и товарами. У заказов также есть доставка. Во время доставки товара водитель может меняться. Это говорит о том, что у доставщика есть более одного заказа на доставку и у заказа может быть несколько курьеров. Также между сущностями “Заказ” и “Курьер” отношение “Многие ко многим”. Отношения многие ко многим выносятся в отдельные таблицы, с двумя первичными ключами.

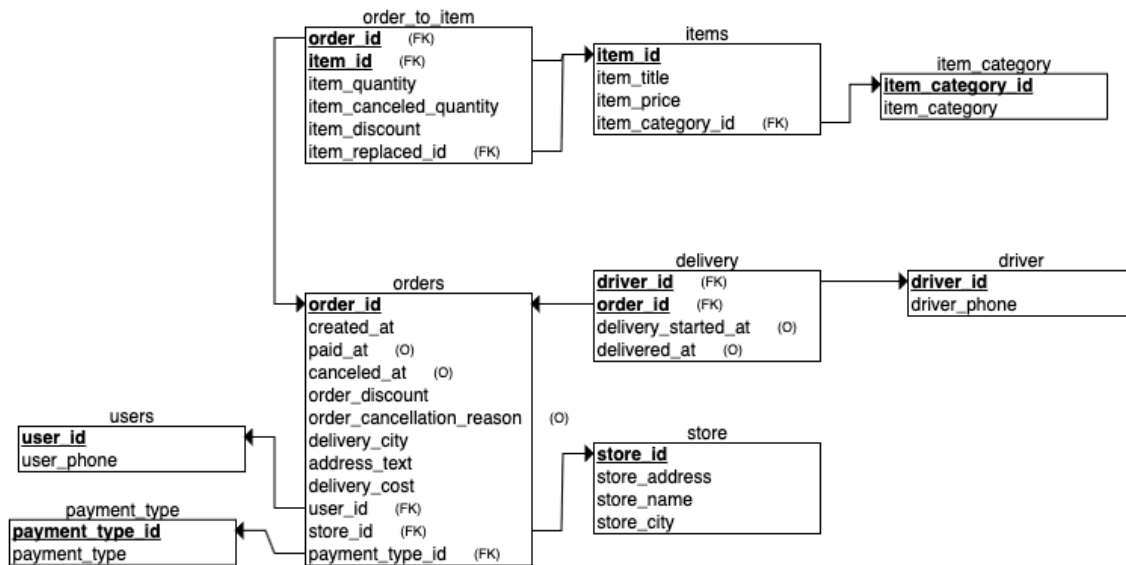
4. **Таблица items** - Хранит в себе информацию о товаре. Из исходных данных имеем идентификатор товара, название товара, цену товара. Также у товара есть категория. Для нормализации данных и производительности хранения, справочник с категорией товара был вынесен отдельно в таблицу item_category, а таблица item ссылается на ключ из таблицы.
5. **Таблица item_category** - Справочник с данными о категории товара. Хранит серийный номер (предоставляется владельцем или с помощью признака serial) и название категории.
6. **Таблица orders** - Хранит в себе все данные о заказе, которые не зависят от других атрибутов. Заказ состоит из идентификатора, даты создания заказа, даты оплаты заказа, даты отмены заказа, скидки на заказ, причину отмены заказа,

идентификатор пользователя, который создает заказ, идентификатор магазина, из которого поступает заказ, тип оплаты, стоимость доставки и адрес доставки. Скидка может быть персональная или постоянная, конкретной информации нет, поэтому не храним ее отдельно. Адрес доставки, также как и стоимость доставки указаны у всех заказов, даже отмененных до доставки. Для хранения этой информации, указываем ее в заказе. Также адрес доставки делим на город и адрес. Тип оплаты вынесено в отдельную таблицу как справочное значение.

7. **Таблица `payment_type`** - справочное значение. Состоит из идентификатора (выделено нами) и название типа оплаты
8. **Таблица `order_to_item`** - Хранит в себе информацию о товарах в каждом заказе. Для хранения данных в таблице в качестве ключа достаточно `order_id` и `item_id`. Для каждого товара в заказе указывается количество товара, количество отменённого товара и идентификатор замененного, как внешний ключ к таблице товаров, также хранится скидка на товар. Скидка может быть индивидуальная, по промокоду, по акции. Информации по видам скидок нет, поэтому храним простой атрибут.
9. **Таблица `delivery`** - Хранит факт доставки, а именно отношение между заказом и курьером со временем начала доставки и временем завершения доставки.

Итог: Модель данных представленная на реляционной схеме стремится к 3 нормальной форме, где отсутствуют транзитивные связи внутри сущности. Наличие таких связей выносятся в отдельные таблицы. Такая схема позволяет сохранять данные о заказах вне зависимости от изменений в товарах, пользователях или адресах магазинов.

Реляционная схема данных



DDL скрипты

```
CREATE TABLE users
```

```
(
  user_id INT NOT NULL,
  user_phone VARCHAR NOT NULL,
  PRIMARY KEY (user_id)
);
```

```
CREATE TABLE driver
```

```
(
  driver_id INT NOT NULL,
  driver_phone VARCHAR NOT NULL,
  PRIMARY KEY (driver_id)
);
```

```
CREATE TABLE store
```

```
(
  store_id INT NOT NULL,
  store_address VARCHAR NOT NULL,
  store_name VARCHAR NOT NULL,
  store_city VARCHAR NOT NULL,
  PRIMARY KEY (store_id)
);
```

```
CREATE TABLE payment_type
```

```
(
  payment_type_id INT NOT NULL,
  payment_type VARCHAR NOT NULL,
  PRIMARY KEY (payment_type_id)
);
```

```
);
```

```
CREATE TABLE item_category
(
  item_category_id INT NOT NULL,
  item_category VARCHAR NOT NULL,
  PRIMARY KEY (item_category_id)
);
```

```
CREATE TABLE orders
(
  order_id INT NOT NULL,
  created_at TIMESTAMP NOT NULL,
  paid_at TIMESTAMP,
  canceled_at TIMESTAMP,
  order_discount FLOAT NOT NULL,
  order_cancellation_reason VARCHAR,
  delivery_city VARCHAR NOT NULL,
  address_text VARCHAR NOT NULL,
  delivery_cost FLOAT NOT NULL,
  user_id INT NOT NULL,
  store_id INT NOT NULL,
  payment_type_id INT NOT NULL,
  PRIMARY KEY (order_id),
  FOREIGN KEY (user_id) REFERENCES users(user_id),
  FOREIGN KEY (store_id) REFERENCES store(store_id),
  FOREIGN KEY (payment_type_id) REFERENCES payment_type(payment_type_id)
);
```

```
CREATE TABLE items
(
  item_id INT NOT NULL,
  item_title VARCHAR NOT NULL,
  item_price FLOAT NOT NULL,
  item_category_id INT NOT NULL,
  PRIMARY KEY (item_id),
  FOREIGN KEY (item_category_id) REFERENCES item_category(item_category_id)
);
```

```
CREATE TABLE order_to_item
(
  item_quantity FLOAT NOT NULL,
  item_canceled_quantity FLOAT NOT NULL,
  item_discount FLOAT NOT NULL,
  order_id INT NOT NULL,
  item_id INT NOT NULL,
  item_replaced_id INT,
  PRIMARY KEY (order_id, item_id),
```

```
FOREIGN KEY (order_id) REFERENCES orders(order_id),  
FOREIGN KEY (item_id) REFERENCES items(item_id),  
FOREIGN KEY (item_replaced_id) REFERENCES items(item_id)  
);
```

```
CREATE TABLE delivery  
(  
    delivery_started_at TIMESTAMP,  
    delivered_at TIMESTAMP,  
    driver_id INT NOT NULL,  
    order_id INT NOT NULL,  
    PRIMARY KEY (driver_id, order_id),  
    FOREIGN KEY (driver_id) REFERENCES driver(driver_id),  
    FOREIGN KEY (order_id) REFERENCES orders(order_id)  
);
```

Основные этапы парсинга

Parquet_parsing_final.ipynb

Основная идея данной части работы состояла в том, что необходимо было создать скрипт для ненормализованных данных в виде однотипной таблицы, распределенной по нескольким файлам (так называемые чанки). Данный скрипт должен считывать каждую часть и разбивать ее на нормализованные по 3НФ таблицы в соответствии с ранее определенной даталогической моделью. Можно выделить следующие этапы работы с parquet-файлами:

- Создание таблицы user
- Создание таблицы driver
- Создание таблицы store
- Мэппинг типов оплаты и id, создание таблицы payment_type
- Создание таблицы item_category
- Создание таблицы order
- Создание таблицы item
- Создание таблицы orderToItem
- Создание таблицы delivery
- Анализ причин отмены доставки

Таблица **user** создавалась по атрибутам user_id и user_phone. Выбирались уникальные пары (user_id, user_phone), где user_id - целочисленный id пользователя, а user_phone - строка, содержащая номер телефона.

Таблица User: 8213 записей	
user_id	user_phone
102	+7 509 928 4364
167	+7 (584) 892-52-60
177	8 491 793 0174
183	8 (335) 479-67-72
217	8 680 834 90 85

Таблица **driver** создавалась по атрибутам driver_id и driver_phone. Выбирались уникальные пары (driver_id, driver_phone), где driver_id - целочисленный id доставщика, а driver_phone - строка, содержащая его номер телефона.

Таблица driver: 390 записей	
driver_id	driver_phone
1001	8 (678) 437-9608
1002	8 323 236 57 28
1003	+7 053 769 9432
1004	+7 001 327 81 76
1005	+7 (221) 987-6249

only showing top 5 rows

Таблица **store** создавалась по атрибутам store_id, store_address и store_name. При этом дополнительно из атрибута store_address был извлечен город. Итоговая таблица содержит уникальные сочетания (store_id, store_address и store_name)

Таблица store: 3 записей			
store_id	store_address	store_city	store_name
1	Сберегайка у дома...	Орел	Сберегайка у дома
2	Сберегайка МАКСИМ...	Москва	Сберегайка МАКСИМУМ
3	Сберегайка у дома...	Тула	Сберегайка у дома

Таблица **payment_type** создавалась по атрибутам payment_type_id и payment_type. Поскольку у способов оплаты не было своего уникального id, был выполнен предварительный маппинг типов оплаты и целочисленных индексов. Далее была составлена итоговая таблица из уникальных пар (payment_type_id, payment_type).

Таблица payment_type: 2 записей	
payment_type_id	payment_type
0	Постоплата курьеру
1	Сразу

Таблица **item_category** создавалась по атрибутам item_category_id и item_category. Поскольку у категорий не было своего уникального id, для них были созданы целочисленные индексы. Далее была составлена итоговая таблица из уникальных пар (item_category_id, item_category).

Таблица item_category: 6 записей

item_category_id	item_category
1	Молочные продукты
2	Мясо
3	Напитки
4	Овощи
5	Фрукты
6	Хлеб

Таблица **order** создавалась по атрибутам order_id, created_at, paid_at, canceled_at, order_discount, order_cancellation_reason, delivery_cost, address_text, user_id, store_id, delivery_city и payment_type_id.

Таблица Order: 10000 записей

order_id	created_at	paid_at	canceled_at	order_discount	order_cancellation_reason	delivery_cost	address_text	user_id	store_id	delivery_city	payment_type_id
49573	2025-07-03	2025-07-04	2025-07-04	5.0	Ошибка приложения	249.0	Москва, ул. Азовс...	304571	2	Москва	0
42492	2025-09-12	2025-09-12	2025-09-12	10.0	Покупатель отменил	280.0	Москва, бул. Крас...	340715	2	Москва	0
40031	2025-06-07	2025-06-07	2025-06-07	0.0	Проблемы с оплатой	188.0	Орел, ул. Роднико...	230716	1	Орел	0
44287	2024-12-13	2024-12-13	2024-12-13	0.0	Проблемы с оплатой	157.0	Тула, наб. Путева...	344191	3	Тула	0
44754	2025-07-18	2025-07-18	2025-07-18	0.0	Покупатель отменил	110.0	Москва, наб. Фабр...	310220	2	Москва	0
45377	2025-04-26	2025-04-26	2025-04-26	0.0	Покупатель отменил	382.0	Тула, пр. Докучае...	49637	3	Тула	0
40079	2025-02-12	2025-02-12	2025-02-12	0.0	Покупатель отменил	334.0	Москва, наб. Став...	110701	2	Москва	0
49477	2025-07-22	2025-07-22	2025-07-22	0.0	Покупатель отменил	291.0	Москва, пр. Абрик...	47536	2	Москва	0
47385	2025-01-05	2025-01-05	2025-01-05	5.0	Покупатель отменил	165.0	Орел, алл. Фрунзе 74	341477	1	Орел	0
48679	2025-01-22	2025-01-22	2025-01-22	5.0	Ошибка приложения	184.0	Орел, пер. Калуж...	131556	1	Орел	0
41675	2025-09-06	2025-09-06	2025-09-06	10.0	Ошибка приложения	209.0	Орел, бул. Комаро...	311897	1	Орел	0
41392	2025-07-29	2025-07-29	2025-07-29	5.0	Ошибка приложения	317.0	Тула, бул. Алтайс...	75032	3	Тула	0
42957	2025-02-23	2025-02-23	2025-02-23	10.0	Покупатель отменил	263.0	Москва, ш. Щербак...	129656	2	Москва	0
48472	2025-03-28	2025-03-28	2025-03-28	0.0	Покупатель отменил	250.0	Тула, алл. Теплич...	319910	3	Тула	0
40236	2025-11-11	2025-11-11	2025-11-11	10.0	Покупатель отменил	147.0	Тула, бул. Широки...	74787	3	Тула	0
42475	2025-07-16	2025-07-16	2025-07-16	0.0	Покупатель отменил	174.0	Орел, алл. Матрос...	14046	1	Орел	0
48738	2025-01-12	2025-01-12	2025-01-12	20.0	Покупатель отменил	316.0	Орел, наб. Шаумян...	159731	1	Орел	0
41812	2025-03-17	2025-03-17	2025-03-17	20.0	Покупатель отменил	279.0	Орел, ул. Победы 69	182933	1	Орел	0
44984	2025-01-16	2025-01-16	2025-01-16	0.0	Проблемы с оплатой	238.0	Москва, пер. Локо...	203289	2	Москва	0
45206	2025-07-08	2025-07-08	2025-07-08	0.0	Ошибка приложения	172.0	Москва, ш. Братск...	236050	2	Москва	0

only showing top 20 rows

Таблица **item** создавалась по атрибутам item_id, item_title, item_price и item_category_id. Уникальные записи хранятся в этой таблице.

Таблица Item: 18 записей

item_id	item_title	item_price	item_category_id
1	Яблоки	335.0	5
2	Бананы	442.0	5
3	Апельсины	99.0	5
4	Огурцы	470.0	4
5	Помидоры	103.0	4
6	Капуста	580.0	4
7	Молоко	219.0	1
8	Кефир	243.0	1
9	Творог	233.0	1
10	Кола	142.0	3
11	Минеральная вода	153.0	3
12	Сок	109.0	3
13	Свинина	209.0	2
14	Говядина	567.0	2
15	Курица	471.0	2
16	Батон	584.0	6
17	Черный хлеб	332.0	6
18	Булочка	129.0	6

Таблица **orderToItem** создавалась по атрибутам item_id, order_id, item_quantity, item_canceled_quantity, item_replaced_id, item_discount. Она содержит уникальные записи.

Таблица orderToItem: 256515 записей

order_id	item_id	item_quantity	item_canceled_quantity	item_replaced_id	item_discount
40023	4	4.0	0.0	0	5.0
40027	10	13.0	0.0	0	5.0
40047	4	8.0	0.0	0	0.0
40070	8	1.0	0.0	0	0.0
40071	8	3.0	0.0	0	0.0
40083	15	11.0	0.0	0	10.0
40087	16	12.0	0.0	0	10.0
40087	4	8.0	0.0	0	0.0
40093	15	5.0	0.0	0	0.0
40109	3	15.0	0.0	0	0.0
40113	4	6.0	0.0	0	0.0
40138	10	8.0	0.0	0	0.0
40139	13	11.0	0.0	0	10.0
40144	1	12.0	0.0	0	10.0
40157	10	6.0	0.0	0	5.0
40162	5	14.0	0.0	0	0.0
40173	1	11.0	0.0	0	0.0
40181	13	12.0	0.0	0	0.0
40185	14	7.0	0.0	0	0.0
40185	10	12.0	0.0	0	10.0

only showing top 20 rows

Таблица **delivery** создавалась по атрибутам order_id, driver_id, delivery_started_at, delivered_at.

order_id	driver_id	delivery_started_at	delivered_at
40493	1043	2025-08-12 15:23:...	2025-08-12 17:48:...
40553	2084	2025-08-24 20:54:...	2025-08-24 22:35:...
40561	3084	2025-07-25 13:27:...	2025-07-25 14:56:...
40678	1114	2025-01-03 13:00:...	2025-01-03 13:30:...
40810	1127	2025-10-10 23:57:...	2025-10-11 02:29:...
41091	2022	2025-03-02 05:37:...	2025-03-02 06:34:...
41095	3044	2025-06-04 02:42:...	2025-06-04 03:45:...
41272	1114	2024-12-27 11:25:...	2024-12-27 13:18:...
41553	2007	2025-10-14 16:55:...	2025-10-14 19:51:...
41577	3004	2025-06-15 00:09:...	2025-06-15 02:09:...
41829	2108	2025-10-26 23:50:...	2025-10-27 01:38:...
41910	3115	2025-01-31 08:51:...	2025-01-31 09:30:...
42257	1140	2025-05-16 05:21:...	2025-05-16 08:14:...
42724	2102	2025-09-15 22:47:...	2025-09-16 00:22:...
42979	1135	2025-03-21 20:09:...	2025-03-21 22:46:...
42982	1107	2025-11-21 11:43:...	2025-11-21 12:12:...
43391	3008	2025-06-12 20:17:...	2025-06-12 22:20:...
44271	3044	2025-01-23 13:54:...	2025-01-23 16:09:...
44351	2078	2025-04-16 12:51:...	2025-04-16 15:49:...
44557	1095	2025-07-25 18:26:...	2025-07-25 18:59:...

only showing top 20 rows

Помимо этого, был выполнен анализ отмененных заказов, поскольку эти значения в дальнейшем играют важную роль при построении витрин. В частности, обнаружено, что как отмененные, так и не отмененные заказы все имели указанную стоимость. Также были посчитаны различные другие статистики по отмененным заказам (см. витрины и ноутбук)

is_canceled	is_delivered	total_orders	orders_with_delivery_cost	total_delivery_cost	orders_with_positive_cost
0	0	851	851	213008	851
0	1	7659	7659	1915119	7659
1	0	1195	1195	303561	1195
1	1	295	295	73044	295

При первичном анализе интересно было посмотреть на причины отмены доставки, всего их 3:

order_cancellation_reason	total	with_delivery_cost	total_cost
Покупатель отменил	397	397	100333
Проблемы с оплатой	380	380	94437
Ошибка приложения	360	360	90852
NULL	353	353	90983

Загрузка данных

Витрины

В рамках проекта были реализованы витрины данных, предназначенные для формирования аналитической и управленческой отчетности. Они выступают в роли промежуточного аналитического слоя между нормализованной транзакционной моделью данных и конечными потребителями информации (аналитиками, BI-инструментами/отчетами).

Основная цель построения витрин – упрощение работы с данными за счет предварительного расчета ключевых бизнес-метрик и предоставления их в удобном виде. Это позволяет выполнять простые запросы с фильтрацией и агрегацией без необходимости каждый раз обращаться к детализированным таблицам и воспроизводить сложную логику расчетов.

В рамках работы были реализованы две витрины:

- витрина заказов;
- витрина товаров.

Витрина заказов

🔗 `Vitrina_1 final .ipynb`

Цель

Согласно заданию и удобству для пользователя витрина предназначена для анализа заказов в разрезах:

- Год
- Месяц
- День
- Город
- Магазин

Источники данных

Источниками витрины стали следующие исходные таблицы:

- Order – информация о заказах (даты создания, оплаты, отмены, покупатель, магазин, причина отмены, скидка на заказ, стоимость доставки);
- OrderToItem – позиции заказа (товары, количество, отменённое количество, скидки, замены);
- Item – справочник товаров (цены);
- Delivery – информация о доставке (курьеры, факт доставки, даты доставки);
- Store – справочник магазинов (название, город).

Гранулярность

Витрина заказов строится с гранулярностью: 1 строка = 1 день × 1 город × 1 магазин. Такая гранулярность позволяет считать дневные показатели, агрегировать

данные до месяцев/года, фильтровать по магазинам, городам без использования дополнительных джоиннов.

Основные этапы создания

Для начала сформировали промежуточный слой данных на уровне позиции заказа, то есть для каждой позиции рассчитываем фактическое количество товара с учетом отмен (то $item_quantity - item_canceled_quantity$), определяем фактическую цену товара (если товар был заменен - используется цена товара, на который был заменен, иначе - исходная) и учитываем скидки на товар ($item_discount$). Так мы каждую позицию приводим, с которым можно дальше работать.

Далее данные агрегируем на уровень заказа, то есть для каждого заказа рассчитывается:

- Оборот – сумма заказанных товаров с учетом скидок;
- Выручка – сумма оплаченных товаров (учитываются только заказы с заполненным $paid_at$);
- Прибыль – разница между выручкой и расходами (расходы на доставку).

Из таблицы доставок агрегируется следующая информация:

- факт доставки заказа;
- суммарная стоимость доставки;
- количество уникальных курьеров, участвовавших в доставке заказа.

Если в одном заказе участвовало более одного курьера, то такой заказ считается заказом со сменой курьера.

Уже после подготовки данных выполняется агрегация на уровне: даты (год, месяц, день), города, магазина и на этом этапе рассчитываются итоговые показатели витрины заказов.

Рассчитываемые метрики витрины заказов

1. Финансовые метрики
 - a. Оборот – сумма заказанных товаров с учетом скидок;
 - b. Выручка – сумма оплаченных товаров;
 - c. Прибыль – выручка за вычетом расходов на доставку.
2. Метрики заказов
 - a. количество созданных заказов;
 - b. количество доставленных заказов;
 - c. количество отмененных заказов;
 - d. количество отмен после доставки;
 - e. количество отмен по причинам «Ошибка приложения» и «Проблемы с оплатой».
3. Клиентские метрики
 - a. количество уникальных покупателей;
 - b. средний чек;
 - c. количество заказов на покупателя;
 - d. выручка на покупателя.
4. Логистические метрики

- a. количество заказов со сменой курьера;
- b. количество активных курьеров (уникальные курьеры, доставившие заказы в указанный день в данном магазине).

В результате формируется витрина заказов, которая содержит полный набор бизнес-метрик, агрегирована в требуемых аналитических разрезах и позволяет выполнять аналитические запросы без сложной логики соединений и пересчетов.

Витрина реализована с использованием Apache Spark, Spark SQL и DataFrame API и может использоваться в качестве источника данных для построения отчетов, дашбордов и проведения последующего аналитического анализа.

Витрина товаров

Витрина товаров формирует аналитический слой для оценки продаж и спроса на уровне товарных позиций без необходимости повторной обработки сырых транзакционных данных. Витрина предназначена для анализа товаров в разрезах:

- Год
- Месяц
- День
- Город доставки (куда доставляли)
- Магазин
- Категория товара
- Товар

На основе витрины можно рассчитывать ключевые товарные метрики: оборот по товару, количество проданных и отмененных единиц, популярность товаров по периодам.

Источники данных

Источниками витрины стали следующие таблицы:

- order - информация о заказах: дата создания, скидка на заказ, магазин, город доставки, пользователь, стоимость доставки, причина отмены и другие
- orderToItem - позиции заказа: товар, количество, отмененное количество, скидка на товар, замененный товар
- item - справочник товаров: название, цена, категория
- item_category - справочник категорий
- store - справочник магазинов: название, адрес, город магазина.

Гранулярность

Витрина товаров строится с гранулярностью:

1 строка = 1 день x 1 город доставки x 1 магазин x 1 категория x 1 товар.

Выбранная гранулярность дает возможность:

- считать дневные показатели по товарам
- агрегировать данные до месяца/года
- находить топ/анти-топ товары в разрезах города и магазина без дополнительных операций соединения.

Основные этапы создания

Этап 1: Формирование промежуточного слоя `order_item_stage`

Базовый слой собирается на уровне позиции заказа (`order_id` x `item_id`) из таблицы `orderToItem`, к которой присоединяются атрибуты из:

- `order`: `created_at`, `order_discount`, `delivery_city`, `store_id`
- `item`: `item_title`, `item_price`, `item_category_id`
- `item_category`: `item_category`
- `store`: `store_name`, `store_address`.

Далее рассчитываются показатели на уровне позиции.

Шаг 1. Фиксация логики замен

В данных при замене присутствуют две строки:

- заменяемый товар: содержит `item_replaced_id` и имеет отмененное количество
- товар-замена: отдельная строка без `item_replaced_id`.

Чтобы корректно отделить исходный спрос от факта, формируется таблица `replacement_ids`:

- берём строки, где `item_replaced_id != 0`
- сохраняем связь (`order_id` - `replacement_item_id`).

После этого строка товара-замены определяется как `is_replacement_item = True`, если `item_id` входит в `replacement_ids` для данного заказа.

Шаг 2. Расчет количественных метрик

- `qty = item_quantity`
- `canceled_quantity = item_canceled_quantity`
- `net_quantity = max(qty - canceled_quantity, 0)`
- `gross_quantity = 0`, если строка - товар-замена, иначе `gross_quantity = qty`.

Таким образом:

- `gross` отражает исходно заказанное количество
- `net` отражает итоговую корзину после отмен.

Шаг 3. Денежные метрики (оборот товара)

Денежные показатели считаются от `net_quantity`, так как финансовый результат должен отражать фактический итог:

- `amount_before_discounts = net_quantity * item_price`
- `amount_after_item_discount = net_quantity * item_price * (1 - item_discount/100)`
- `amount_after_order_discount = amount_after_item_discount * (1 - order_discount / 100)`

Этап 2: Агрегация в витрину `dm_mart`

После подготовки `order_item_stage` данные агрегируются на уровень:

год x месяц x день x город доставки x магазин x категория x товар.

На этом уровне считаются итоговые показатели витрины.

Рассчитываемые метрики витрины товаров:

1. Количественные метрики:
 - a. Количество заказанных единиц товара
 - b. Количество отмененных единиц товара
 - c. Количество фактически проданных/оставшихся единиц товара
2. Метрики заказов:
 - a. Количество заказов с товаром
 - b. Количество заказов с отменой товара
3. Финансовые метрики:
 - a. Оборот до скидок
 - b. Оборот после скидки товара
 - c. Оборот после скидки заказа
4. Определение популярных / непопулярных товаров
 - a. самый популярный товар за день/месяц/год (товар с максимальным заказанным количеством или с максимальной выручкой после применения скидок)
 - b. самый непопулярный товар за день/месяц/год (по аналогии)

В результате формируется агрегированная в требуемых разрезах витрина товаров, которая может использоваться для формирования отчётов и дашбордов. Витрина реализована с использованием Apache Spark и Spark DataFrame API.